

中山大学 计算机学院本科生实验报告

(2025/2026学年)

课程名称	并行程序设计与算法 (实验)	批改人	
实验	Lab 4 - Pthreads 条件变量与蒙特卡洛算法	专业 (方向)	计算机科学与技术 (人工智能)
学号	23336103	姓名	雷颜玮
Email	leiyanwei2005@163.com	完成日期	2026年5月2日

1. 实验要求与目的

使用 Pthreads 编写多线程程序，分别完成两个典型的并行问题：

- 一元二次方程求解**：体会多线程环境中任务间存在的依赖关系，使用互斥锁（Mutex）与条件变量（Condition Variable）实现前置计算与后置依赖的线程同步。
- 蒙特卡洛方法求圆周率**：理解随机采样的统计计算方法，并使用 Pthreads 基于多核CPU开展数据级并行（Data Parallelism）优化。

2. 一元二次方程并行求解

2.1 算法设计与并行逻辑

使用求根公式并行求解一元二次方程 $ax^2 + bx + c = 0$ ：

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

公式的求解中存在强依赖关系：必须先求出判别式 $\Delta = b^2 - 4ac$ ，才能分别求解 x_1 和 x_2 。因此程序中通过三个线程调度：

- Thread 1 (Delta)**：计算 Δ ，完成后将结果存入全局变量，将 `delta_ready` 设为 `true`，并通过 `pthread_cond_broadcast` 唤醒所有正在等待的线程。
- Thread 2 & Thread 3 (x1 & x2)**：首先使用 `pthread_cond_wait` 挂起，等待 `delta_ready` 条件达成。被唤醒后读取 Δ ，接着各自独立地完成 x_1 或 x_2 的后续四则运算。

2.2 核心代码实现

利用互斥锁保护共享状态 `delta_ready` 和 `delta_val`，配合条件变量 `cond_delta` 实现等待-唤醒机制。

```
// 线程1: 计算 Delta, 并在完成后广播唤醒等待它的线程
void* calc_delta(void* arg) {
    double d = b * b - 4 * a * c;

    pthread_mutex_lock(&mutex);
    delta_val = d;
    delta_ready = true;
    pthread_cond_broadcast(&cond_delta); // 唤醒所有等待的线程
    pthread_mutex_unlock(&mutex);
}
```

```

        return nullptr;
    }

    // 线程2: 计算 x1, 依赖 Delta 的结果 (x2的计算逻辑同理)
    void* calc_x1(void* arg) {
        pthread_mutex_lock(&mutex);
        while (!delta_ready) {
            pthread_cond_wait(&cond_delta, &mutex); // 挂起并等待条件满足
        }
        double d = delta_val;
        pthread_mutex_unlock(&mutex);

        if (d >= 0) {
            x1_val = (-b + sqrt(d)) / (2 * a);
        }
        return nullptr;
    }
}

```

2.3 测试输出与性能讨论

测试输出:

```

Input a, b, c [-100, 100]: 1.0 -3.0 2.0
Roots: x1 = 2, x2 = 1
Time consumed t: 0.000327085 s

```

性能讨论:

实验逻辑上成功验证了条件变量完成细粒度同步的正确性。但从执行效能讲, 求解极少量标量浮点计算耗时微乎其微。相反, 拉起多个线程和执行同步锁的系统开销 (通常在数十微秒到毫秒级) 大于其串行执行的时间。为了产生有效的并行增益, 需要扩展至**高维批处理计算场景** (大规模并行地解决多个独立方程的情况), 此时才能摊薄线程初始化的代价。

3. 蒙特卡洛求 π 值

3.1 算法设计与数据并行分配

蒙特卡洛求 π 是一个完全没有数据依赖的 "Embarrassingly parallel" 问题。根据给定总采样点 n , 每个子线程独立分配均分的 $\lceil \frac{n}{\text{num_threads}} \rceil$ 次试验, 独立生成二维随机坐标进行投点并各自统计落在圆内的数目。各线程完全结束后, 使用 PTHREAD_MUTEX_LOCK 安全合并其累计的局部总数 m , 从而无冲突地进行概率近似。

3.2 核心代码实现

核心代码采用了安全的 `rand_r()` 避免伪随机数生成器的隐式锁争用, 并使用局部变量 `local_m` 进行投点结果累积以消除循环内的互斥锁开销, 最后仅在归约时加全局锁。

```

void* monte_carlo(void* arg) {
    long long thread_id = (long long)arg;
    long long local_n = total_n / num_threads; // 加上尾数分配逻辑(省略)
    long long local_m = 0; // 线程局部累加器, 避免内存写竞争

    // 设定各自独立线程的随机数种子

```

```
    unsigned int seed = chrono::system_clock::now().time_since_epoch().count() +
    thread_id;

    for (long long i = 0; i < local_n; ++i) {
        double x = (double)rand_r(&seed) / RAND_MAX;
        double y = (double)rand_r(&seed) / RAND_MAX;
        if (x * x + y * y <= 1.0) {
            local_m++;
        }
    }

    // 局部结果汇总至全局总数，加锁保护临界区
    pthread_mutex_lock(&m_mutex);
    total_m += local_m;
    pthread_mutex_unlock(&m_mutex);

    return nullptr;
}
```

3.3 不同线程下的加速比评测

为了更直观地衡量线程数的增加带来的加速效果，编写了自动化测试脚本 `run_experiments.py`，并将 n 固定为 100,000,000 进行了多次测试，结果如下：

Threads	Time (s)	Speedup	Efficiency
1	0.581670	1.00x	100.00%
2	0.292288	1.99x	99.50%
4	0.158956	3.66x	91.48%
8	0.084002	6.92x	86.56%
16	0.058050	10.02x	62.63%

性能讨论：

- 1. **线性加速能力：**当线程数从 1 扩展至 4 时，加速比分别为 1.99x 和 3.66x，体现出优异的近线性扩展性。由于多核 CPU 的独立内核被彻底利用，该计算模式实现了极高的并发效率。
- 2. **硬件饱和与收益递减：**当线程数继续攀升（如增加至 16 时），加速比上升至 10.02x，但效率（Efficiency）回落至 62%。这是遇到了 CPU 的物理和超线程核心的天花板调度以及互斥累加锁最后的开销瓶颈。总体说明基于数据划分的粗粒度方法非常适配于随机采样场景。